

Scelte progettuali e costi computazionali

Metodo delle correnti di maglia per la risoluzione di circuiti elettrici

Relazione tecnica di supporto alla discussione orale del progetto

1. Notazione e ipotesi

In tutto il documento indichiamo con **n** il numero di nodi del circuito, con **m** il numero di lati (cioè di componenti: $m = m_R + m_V$, resistori più generatori) e con **k** = **m** - **n** + **1** il numero di maglie indipendenti, cioè la dimensione dello spazio dei cicli. L'uguaglianza $k = m - n + 1$ vale per un grafo *connesso*; in generale è $m - n + c$, con *c* numero di componenti connesse. Lavoriamo sotto le ipotesi della specifica: circuito connesso, soli resistori e generatori ideali di tensione, nessun componente in parallelo. Per i circuiti "semplici" oggetto del progetto *n*, *m*, *k* sono dell'ordine delle decine, quindi i tempi assoluti sono trascurabili: le stime che seguono servono a motivare le scelte e a capire *quale parte domina* al crescere della dimensione.

2. Scelte strutturali

2.1 Circuito come grafo non orientato

Il metodo delle correnti di maglia è naturalmente un problema su grafo: i nodi sono i punti di collegamento, i lati sono i componenti e le maglie sono i cicli fondamentali. Abbiamo quindi costruito la topologia con la classe `undirected_graph`, che mantiene *due* strutture in parallelo: una **mappa di adiacenza** (`map<T, set<T>>`) e un **vettore di lati numerati** (`vector<undirected_edge>`). La motivazione è che servono due cose diverse: l'adiacenza per le visite (DFS, BFS, Dijkstra) e una *numerazione stabile* dei lati per indicizzare righe e colonne delle matrici *B* e *R* e dei vettori di incidenza di De Pina. Tenere entrambe evita di ricostruire continuamente l'una dall'altra, al prezzo di una piccola ridondanza di memoria ($O(n + m)$).

2.2 Lato con nodi ordinati

La classe `undirected_edge` ordina sempre i due estremi (from minore di to). Questo dà tre vantaggi con una sola scelta: (i) (u,v) e (v,u) diventano lo stesso lato, evitando duplicati; (ii) il lato è usabile come chiave in `map/set` grazie all'ordinamento lessicografico; (iii) fissa un *verso di percorrenza canonico* dal nodo minore al maggiore, esattamente la convenzione richiesta dalla specifica per stabilire i segni in *B*.

2.3 Separazione topologia / semantica

La classe `circuit_graph` usa la composizione: incapsula un `undirected_graph` (la topologia) e vi associa i `component` (la semantica elettrica: tipo, valore, polarità). I resistori e i generatori sono inoltre tenuti in due liste separate. La motivazione è duplice: separare le responsabilità (il grafo non sa nulla di Volt e Ohm) e poter riusare quasi integralmente il codice dei grafi sviluppato a esercitazione. La separazione resistori/generatori rispecchia direttamente la teoria: i primi alimentano la matrice *B* e *R*, i secondi il termine noto *v*.

2.4 Algebra lineare confinata

Tutte le operazioni numeriche (prodotti matrice-vettore, gradiente coniugato, eventuale numero di condizionamento) sono raccolte in `eigen_support` sopra la libreria `Eigen`. La scelta isola la dipendenza esterna in un solo punto e tiene il resto del codice indipendente dalla libreria algebrica.

3. Scelte algoritmiche

3.1 Due metodi per i cicli fondamentali

La specifica richiede entrambe le alternative, che abbiamo implementato. `find_cycles` costruisce un albero di supporto con una DFS, calcola il coalbero $C = G - T$ e chiude un ciclo per ogni lato del coalbero (cammino nell'albero ricostruito con Dijkstra a pesi unitari). È semplice e veloce, ma i cicli ottenuti non sono necessariamente minimi. `de_pina` applica l'algoritmo di De Pina e restituisce una base di *cicli minimi*: per ogni vettore di base S_i cerca il ciclo di peso minimo con intersezione dispari con S_i , usando un grafo "liftato" (segnato) e i cammini minimi su di esso. È più costoso (vedi §5) ma produce maglie più naturali da interpretare.

3.2 Costruzione del sistema

Dai cicli si assemblano la matrice di incidenza B (`build_B`, segni dati dal verso di percorrenza tramite `plus_minus`), la matrice diagonale delle resistenze R (`build_R`) e il termine noto v (`build_v`, segni dei generatori tramite `gen_sign`). Si risolve poi $B^T R B i = v$, da cui le tensioni $v_R = R B i$.

3.3 Risoluzione: gradiente coniugato

La matrice $A = B^T R B$ è **simmetrica e definita positiva** quando R è diagonale positiva (resistenze positive) e B ha rango pieno, cioè quando i cicli sono indipendenti — proprietà garantita da entrambi i metodi di §3.1. Questo giustifica l'uso del **gradiente coniugato** (`gradiente_coniugato`), che converge su sistemi SPD ed è codice già visto a esercitazione. Per sistemi così piccoli andrebbe bene anche una fattorizzazione di Cholesky; il CG è stato preferito per coerenza con il materiale del corso. Nota: l'ipotesi SPD dipende dal rango pieno di B , non solo dalla positività di R .

4. Costi delle operazioni di base

La tabella seguente riporta il costo asintotico delle primitive su grafo. Le voci segnalate con (!) sono quelle che, pur corrette, pesano di più sul costo complessivo e sono i candidati naturali a un'ottimizzazione (vedi §7).

Operazione	Costo	Motivo
<code>neighbors(v)</code>	$O(\log n + \deg v)$	lookup nella mappa + copia dell'insieme dei vicini
<code>add_edge</code>	$O(\log n)$	inserimenti in mappa e insiemi
<code>all_edges()</code>	$O(m \log m)$	ricostruisce un set dal vettore dei lati
<code>all_nodes()</code>	$O(m \log n)$	scorre i lati inserendo i nodi in un set
<code>edge_number(e) (!)</code>	$O(m)$	scansione lineare del vettore dei lati
<code>edge_at(i)</code>	$O(1)$	accesso diretto al vettore
<code>operator- (G-T) (!)</code>	$O(m^2 \log m)$	ricostruisce <code>all_edges()</code> a ogni iterazione esterna
<code>recursive_dfs</code>	$O((n+m) \log n)$	ogni nodo visitato una volta; copia dei vicini
<code>dijkstra</code>	$O((n+m) \log n)$	coda di priorità con <code>std::set</code> ; pesi unitari
<code>get_path_vec</code>	$O(L \log n)$	L = lunghezza del cammino

Tabella 1 — Costo delle primitive su grafo. n = nodi, m = lati, $\deg$ = grado, L = lunghezza cammino.

5. Costo dei due metodi di ricerca delle maglie

DFS + coalbero (find_cycles). Il costo è dominato dal calcolo del coalbero $G - T$, cioè da operator- a $O(m^2 \log m)$, più k cammini minimi sull'albero ($O(k \cdot n \log n)$). Per circuiti con $m = O(n)$ e $k = O(n)$ si ottiene complessivamente circa **$O(n^2 \log n)$** .

De Pina (de_pina). Per ciascuna delle k maglie si esegue una ricerca di ciclo minimo (ciclo_minimo): costruzione del grafo liftato a $O(m^2)$ — penalizzata dalle chiamate a edge_number — più n calcoli di cammino minimo a $O((n+m) \log n)$. Sommando sulle k maglie si ottiene circa **$O(k \cdot n^2 \log n)$** , cioè dell'ordine di **$O(n^3 \log n)$** per $m, k = O(n)$. De Pina è quindi *asintoticamente più costoso* di un fattore circa n rispetto al metodo DFS, in cambio della minimalità dei cicli.

La misura empirica conferma l'analisi. Abbiamo generato reti a scala (ladder) di dimensione crescente e misurato il tempo dei due metodi (build con -O2):

Maglie (circa)	Lati resistivi	De Pina (s)	DFS+coalbero (s)
5	15	0,003	0,002
10	30	0,008	0,002
20	60	0,044	0,003
40	120	0,336	0,005
60	180	1,220	0,009
80	240	2,813	0,016

Tabella 2 — Tempi misurati su reti a scala. Raddoppiando la dimensione, De Pina cresce di circa 8 volte (andamento cubico), il metodo DFS resta quasi piatto.

A 80 maglie De Pina è circa 175 volte più lento. Per i circuiti del progetto la differenza è invisibile (millisecondi), ma se servisse trattare reti grandi il metodo DFS è preferibile, mentre De Pina ha senso quando interessa la minimalità delle maglie.

6. Costo della costruzione e risoluzione del sistema

Fase	Costo	Note
plus_minus / gen_sign	$O(L)$	scorre i nodi del ciclo (L = lunghezza)
build_B	$O(m_r \cdot k \cdot L)$	per ogni resistore e ogni maglia
build_R	$O(m_r)$	diagonale; memoria $O(m_r^2)$ perché densa (migliorabile)
build_v	$O(k \cdot m_v \cdot L)$	per ogni maglia e ogni generatore
$A = B^T R B$	$O(k^2 \cdot m_r)$	prodotto delle matrici
gradiente_coniugato	$O(k^3)$	al più k iterazioni, 2 prodotti A-vettore per iterazione
stampa_risultati	$O(m_r \cdot k)$	$v_r = R B$ i ricomposto per resistore

Tabella 3 — Costo dell'assemblaggio e della risoluzione. m_r = resistori, m_v = generatori.

Complessivamente l'algebra lineare è dell'ordine di $O(n^3)$, confrontabile con il metodo DFS e inferiore a De Pina. Due note di merito sul gradiente coniugato: la nostra forma effettua due prodotti matrice-vettore per iterazione (usa la coniugazione esplicita di A); la forma canonica ne richiede uno solo riusando i prodotti scalari dei residui. La differenza è irrilevante a queste

dimensioni ma è bene saperla motivare.

7. Sintesi e possibili ottimizzazioni

Il collo di bottiglia non è l'algebra lineare ma la **ricerca dei cicli**, e in particolare De Pina. Le ottimizzazioni con il miglior rapporto costo/beneficio, se servisse scalare, sono:

- Sostituire la scansione lineare di `edge_number` ($O(m)$) con una mappa lato $\rightarrow$ indice ($O(1)$): alleggerisce direttamente il grafo liftato e la ricostruzione dei cammini, le parti più pesanti di De Pina.
- In `operator-` calcolare `all_edges()` una sola volta fuori dal ciclo: si scende da $O(m^2 \log m)$ a $O(m \log m)$.
- Memorizzare `R` come vettore diagonale invece che come matrice densa: da $O(m_r^2)$ a $O(m_r)$ in memoria.
- Nel gradiente coniugato usare la forma standard con un solo prodotto matrice-vettore per iterazione.
- Sostituire `dijkstra` a pesi unitari con una BFS ($O(n + m)$) dove i pesi sono tutti 1.

In conclusione, le scelte strutturali privilegiano semplicità e riuso del codice di esercitazione; i costi asintotici sono noti e dominati dalla ricerca dei cicli, con De Pina più oneroso ma più informativo del metodo DFS. Le ottimizzazioni elencate sono mirate e non alterano l'impianto del progetto.